Chapter 17

Task L: Deployment and Production

Deployment is supposed to mark a happy point in the lifetime of our application. It's when we take the code that we've so carefully crafted and upload it to a server so that other people can use it. It's when the beer, champagne, and hors d'oeuvres are supposed to flow. Shortly thereafter, our application will be written about in *Wired* magazine, and we'll be overnight names in the geek community.

The reality, however, is that it often takes quite a bit of up-front planning to pull off a smooth and repeatable deployment of your application.

A bewildering number of options are available for deployment: Ansible, Capistrano, Chef, and Puppet are all popular choices. Covering all of them would be the subject of several books. We're going to focus this chapter on what effectively is the defacto standard for cloud deployment: Docker.

If you're not familiar with Docker images, they're essentially self-contained and portable runtimes that can be deployed by pretty much any cloud-hosting provider. This means you can build and test your deployment locally and then choose your cloud provider later, and even change your mind and move hosts at any time.

At the moment, we've been doing all of our work on one machine, though user interaction with our web server *could* be done on a separate machine. On our machine you've been making use of the Puma web server, SQLite 3, various gems you've installed, and your application code. Your code may or may not